LearningWebsite 技术栈说明（technics）

本文档面向毕业论文撰写，完整说明本项目采用的重要技术栈，并给出关键源码、源码出处及设计解析。

1. 项目技术栈总览

1.1 后端核心栈

- 语言：Python 3.11
- Web 框架：Django 5.1.8
- 模板引擎：Django Templates（服务端渲染 SSR）
- ORM：Django ORM
- 认证系统：Django 内置 `auth`
- 文件存储：Django `Media`（头像、课程封面）
- 数据库：
  - 默认：SQLite（`db.sqlite3`）
  - 可选：MySQL（驱动 `mysqlclient==2.2.7`）

1.2 业务子系统（Apps）

- `accounts`：注册登录、用户资料、角色体系
- `courses`：课程-章节-课时建模、学习进度、内容工作台
- `quiz`：题库、测验提交、错题本
- `forum`：帖子评论、置顶/已解决/状态流转
- `search`：去范式搜索索引与检索聚合
- `analytics`：学习行为统计与 CSV 导出
- `core`：首页、健康检查、演示数据命令

1.3 工程化与质量保障

- 测试：`pytest`、`pytest-django`、Django `TestCase`
- 代码规范：`black`、`isort`、`flake8`
- 覆盖率：`coverage`
- 环境管理：Conda + pip（隔离环境 `learningwebsite`）

1.4 依赖清单出处

源码出处：`requirements.txt`

```
Django==5.1.8
mysqlclient==2.2.7
pillow==11.1.0
pytest==8.3.5
pytest-django==4.11.1
black==25.1.0
isort==6.0.1
flake8==7.1.2
coverage==7.8.0
```

解析：

- `Django` 构成全部 Web 能力基础。
- `mysqlclient` 仅用于 MySQL 可选部署。
- `pillow` 支撑图片字段（头像/封面）的处理。
- `pytest*` 与 Django `TestCase` 共同支撑回归验证。
- `black/isort/flake8/coverage` 构成质量工具链。

---

2. 系统架构设计

2.1 分层结构

本项目采用典型 Django 分层：

- URL 层：路由分发
- View 层：业务编排、权限校验
- Model 层：领域建模、约束与索引
- Template 层：页面渲染
- Command/Signal 层：离线任务与事件驱动索引

2.2 根路由聚合

源码出处：`config/urls.py`

```
urlpatterns = [
    path("", include("core.urls")),
    path("admin/", admin.site.urls),
    path("accounts/", include("accounts.urls")),
    path("courses/", include("courses.urls")),
    path("quiz/", include("quiz.urls")),
    path("forum/", include("forum.urls")),
    path("search/", include("search.urls")),
    path("analytics/", include("analytics.urls")),
]
```

解析：

- 根路由只做“模块装配”，业务路由下沉到各 app。

- 这种结构便于后续迭代（新增模块无需改动核心逻辑）。

---

3. 环境配置与可部署性

3.1 多数据库配置（SQLite / MySQL）

源码出处：`config/settings.py`

```
if os.getenv("DB_ENGINE", "sqlite").lower() == "mysql":
    DATABASES = {
        "default": {
            "ENGINE": "django.db.backends.mysql",
            "NAME": os.getenv("DB_NAME", "learningwebsite"),
            "USER": os.getenv("DB_USER", "root"),
            "PASSWORD": os.getenv("DB_PASSWORD", ""),
            "HOST": os.getenv("DB_HOST", "127.0.0.1"),
            "PORT": os.getenv("DB_PORT", "3306"),
            "OPTIONS": {"charset": "utf8mb4"},
        }
    }
else:
    DATABASES = {
        "default": {
            "ENGINE": "django.db.backends.sqlite3",
            "NAME": BASE_DIR / "db.sqlite3",
        }
    }
```

解析：

- 开发期默认 SQLite，降低环境门槛。
- 通过环境变量无缝切换 MySQL，满足生产部署需要。
- `utf8mb4` 保障中文与 emoji 兼容。

3.2 国际化与时区

源码出处：`config/settings.py`

```
LANGUAGE_CODE = "zh-hans"
TIME_ZONE = "Asia/Shanghai"
USE_I18N = True
USE_TZ = True
```

解析：

- 前端与系统默认中文语境，适配论文与答辩展示。
- 统一时区避免统计/日志时间错位。

---

4. 用户系统与权限模型

4.1 用户扩展与角色体系

源码出处：`accounts/models.py`

```
class UserRole(models.TextChoices):
    STUDENT = "student", "学生"
    TEACHER = "teacher", "教师"
    ADMIN = "admin", "管理员"

class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE, related_name="profile")
    role = models.CharField(max_length=20, choices=UserRole.choices,
default=UserRole.STUDENT, db_index=True)
```

解析：

- 在 Django `User` 外通过 `OneToOne` 扩展业务字段，兼顾灵活性与可维护性。
- 角色以枚举约束，避免魔法字符串。

4.2 信号确保 Profile 自动创建

源码出处：`accounts/models.py`

```
@receiver(post_save, sender=User)
def create_or_update_user_profile(sender, instance, created, **kwargs):
    if created:
        UserProfile.objects.create(user=instance)
        return
    UserProfile.objects.get_or_create(user=instance)
```

解析：

- 保证每个用户均有 `profile`，减少业务层空判断。
- 提升数据一致性，避免注册后资料页异常。

4.3 注册表单中的业务校验

源码出处：`accounts/forms.py`

```
def clean_email(self):
    email = self.cleaned_data["email"].strip().lower()
    if User.objects.filter(email__iexact=email).exists():
        raise forms.ValidationError("该邮箱已被使用。")
    return email
```

解析：

- 邮箱统一小写并做唯一性校验。

- `iexact` 避免大小写绕过重复检查。

---

5. 课程子系统（核心业务域）

5.1 数据模型：课程-章节-课时

源码出处：`courses/models.py`

```
class Course(models.Model):
    title = models.CharField(max_length=200, db_index=True)
    slug = models.SlugField(max_length=220, unique=True, blank=True)

class Chapter(models.Model):
    course = models.ForeignKey(Course, on_delete=models.CASCADE, related_name="chapters")
    order_no = models.PositiveIntegerField(default=1)
    class Meta:
        constraints = [models.UniqueConstraint(fields=["course", "order_no"],
name="uniq_chapter_order_per_course")]

class Lesson(models.Model):
    chapter = models.ForeignKey(Chapter, on_delete=models.CASCADE, related_name="lessons")
    order_no = models.PositiveIntegerField(default=1)
    class Meta:
        constraints = [models.UniqueConstraint(fields=["chapter", "order_no"],
name="uniq_lesson_order_per_chapter")]
```

解析：

- 通过层级外键表达课程内容结构。
- 序号唯一约束保证同一父级下教学顺序不冲突。

5.2 slug 自动生成与去重

源码出处：`courses/models.py`

```
def _build_unique_slug(self, seed: str | None = None) -> str:
    base = slugify(seed or self.title)[:200] or "course"
    candidate = base
    suffix = 1
    while Course.objects.filter(slug=candidate).exclude(pk=self.pk).exists():
        candidate = f"{base}-{suffix}"
        suffix += 1
    return candidate
```

解析：

- 自动路由 slug，兼顾 SEO 与可读性。
- 通过后缀递增确保唯一。

5.3 可见性策略（学生/教师/管理员差异）

源码出处：`courses/views.py`

```
def _visible_courses(user):
    if not user.is_authenticated:
        return Course.objects.filter(status=CourseStatus.PUBLISHED)
    if user.is_staff:
        return Course.objects.all()
    if _is_manager(user):
        return Course.objects.filter(Q(status=CourseStatus.PUBLISHED) | Q(created_by=user))
    return Course.objects.filter(status=CourseStatus.PUBLISHED)
```

解析：

- 统一封装“可见性”，避免权限逻辑散落在各视图。
- 学生只看已发布，教师可看自己创建内容，管理员看全量。

5.4 学习进度模型与索引

源码出处：`courses/models.py`

```
class LearningProgress(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE,
related_name="learning_progress")
    lesson = models.ForeignKey(Lesson, on_delete=models.CASCADE,
related_name="learning_progress")
    completed = models.BooleanField(default=False)

    class Meta:
        constraints = [models.UniqueConstraint(fields=["user", "lesson"],
name="uniq_learning_progress_user_lesson")]
        indexes = [
            models.Index(fields=["user", "completed"], name="idx_learning_user_completed"),
            models.Index(fields=["lesson", "completed"],
name="idx_learning_lesson_completed"),
        ]
```

解析：

- 每个用户在每个课时仅一条进度记录，防止重复。
- 为统计场景增加组合索引，提升仪表盘查询效率。

---

6. 内容治理与审计日志

6.1 审计日志模型

源码出处：`courses/models.py`

```
class ContentAuditLog(models.Model):
    target_type = models.CharField(max_length=20, choices=AuditTargetType.choices,
db_index=True)
    target_id = models.PositiveIntegerField(db_index=True)
    action = models.CharField(max_length=50, db_index=True)
    message = models.CharField(max_length=255)
    payload = models.JSONField(default=dict, blank=True)
```

解析：

- 记录“谁在何时对什么对象做了什么操作”。
- `payload` 可扩展状态变更细节，利于追踪与审计。

6.2 日志写入封装

源码出处：`courses/audit.py`

```
def log_content_action(*, actor, target_type, target_id, action, message, course=None,
chapter=None, lesson=None, payload=None):
    return ContentAuditLog.objects.create(
        actor=actor,
        course=course,
        chapter=chapter,
        lesson=lesson,
        target_type=target_type,
        target_id=target_id,
        action=action,
        message=message,
        payload=payload or {},
    )
```

解析：

- 将日志写入统一收口，减少重复代码。
- 后续替换为异步日志或消息队列时改造成本低。

7. 测验系统设计

7.1 题目模型与数据校验

源码出处：`quiz/models.py`

```
class Question(models.Model):
    question_type = models.CharField(max_length=20, choices=QuestionType.choices,
default=QuestionType.SINGLE_CHOICE)
    options = models.JSONField(default=list, blank=True)
    correct_answer = models.JSONField(default=list)

    def clean(self):
        if not isinstance(self.correct_answer, list):
            raise ValidationError({"correct_answer": "correct_answer 必须是列表类型。"})
```

解析：

- 采用 JSON 结构支持单选/多选/判断/简答的统一存储。
- `clean()` 强制结构合法，降低脏数据风险。

7.2 提交评测算法

源码出处：`quiz/views.py`

```
def _evaluate_submission(questions, post_data):
    for question in questions:
        expected = _normalize_answer_list(question.correct_answer)
        user_answer = _normalize_answer_list(_extract_user_answer(question, post_data))
        is_correct = user_answer == expected
```

解析：

- 先归一化后比较，提升容错性（如大小写、空格）。
- 评测输出 `result_rows`，同时供“结果页展示 + 入库”复用。

7.3 事务化持久化（提交+答案+错题）

源码出处：`quiz/views.py`

```
with transaction.atomic():
    submission = QuizSubmission.objects.create(...)
    QuizAnswer.objects.create(...)
    WrongQuestion.objects.get_or_create(...)
```

解析：

- 使用事务保证一次提交内多表写入原子性。
- 避免“提交有记录但答案/错题缺失”的中间状态。

7.4 错题本机制

源码出处：`quiz/models.py`

```
class WrongQuestion(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE, related_name="wrong_questions")
    question = models.ForeignKey(Question, on_delete=models.CASCADE,
related_name="wrong_users")
    wrong_count = models.PositiveIntegerField(default=1)
    resolved = models.BooleanField(default=False)
```

解析：

- 维护“错题次数 + 是否解决”，支持精准重练。
- 通过 `resolved` 实现错题闭环管理。

---

8. 论坛系统设计

8.1 帖子状态机与元数据

源码出处：`forum/models.py`

```
class ForumPostStatus(models.TextChoices):
    PUBLISHED = "published", "已发布"
    HIDDEN = "hidden", "已隐藏"
    DELETED = "deleted", "已删除"

class ForumPost(models.Model):
    is_pinned = models.BooleanField(default=False)
    is_solved = models.BooleanField(default=False)
    view_count = models.PositiveIntegerField(default=0)
```

解析：

- 状态字段支持内容审核场景。
- `is_pinned/is_solved` 面向教学社区问答运营。

8.2 阅读计数原子更新

源码出处：`forum/views.py`

```
ForumPost.objects.filter(id=post.id).update(view_count=F("view_count") + 1)
```

解析：

- 使用 `F` 表达式避免并发场景下读写覆盖。
- 保证浏览量统计准确性。

8.3 评论树约束

源码出处：`forum/models.py`

```
def clean(self):
    if self.parent_id and self.parent_id == self.pk:
        raise ValidationError({"parent": "评论不能回复自身。"})
    if self.parent_id and self.parent.post_id != self.post_id:
        raise ValidationError({"parent": "父评论必须属于同一帖子。"})
```

解析：

- 避免非法回复关系（自回复/跨帖回复）。
- 提升讨论结构一致性。

---

9. 搜索系统设计（去范式索引）

9.1 索引文档模型

源码出处：`search/models.py`

```
class SearchDocument(models.Model):
    source_type = models.CharField(max_length=30, choices=SearchSourceType.choices,
db_index=True)
    source_id = models.PositiveBigIntegerField(db_index=True)
    title = models.CharField(max_length=255, db_index=True)
    body = models.TextField(blank=True)
    keywords = models.CharField(max_length=255, blank=True)
    metadata = models.JSONField(default=dict, blank=True)
    is_active = models.BooleanField(default=True, db_index=True)
```

解析：

- 不是直接跨业务表全文检索，而是维护独立搜索索引表。
- 查询更可控，能跨课程/课时/帖子/题目统一检索。

9.2 Upsert 索引策略

源码出处：`search/indexing.py`

```
SearchDocument.objects.update_or_create(
    source_type=source_type,
    source_id=source_id,
    defaults=defaults,
)
```

解析：

- 使用幂等 upsert，重复写入不产生脏重复数据。

- 配合唯一约束 `uniq_search_source` 保证索引一致性。

9.3 信号驱动增量索引

源码出处：`search/signals.py`

```
@receiver(post_save, sender=Course)
def index_course_on_save(sender, instance: Course, **kwargs):
    index_course(instance)
```

解析：

- 业务数据变更后自动同步搜索索引。
- 支撑“近实时可搜索”，减少手工重建频率。

9.4 全量重建命令

源码出处：`search/management/commands/rebuild_search_index.py`

```
SearchDocument.objects.all().delete()
for course in Course.objects.select_related("created_by").iterator():
    index_course(course)
```

解析：

- 提供离线重建兜底能力，适合初始化或数据修复。
- `iterator()` 减少大数据量情况下内存压力。

---

10. 学习分析系统（Analytics）

10.1 双视角统计：学生 vs 管理员

源码出处：`analytics/views.py`

```
def index(request):
    if request.user.is_staff:
        context = _build_staff_context()
    else:
        context = _build_student_context(request.user)
```

解析：

- 同一路由按角色切换统计维度。
- 学生看个人进度，管理员看平台全局指标。

10.2 关键统计方法（ORM 聚合）

源码出处：`analytics/views.py`

```
submission_summary = submissions_qs.aggregate(avg_accuracy=Avg("accuracy"))
completion_rate = round((completed_lessons / total_lessons) * 100, 2) if total_lessons else
0
```

解析：

- ORM 聚合直接在数据库层完成，减少 Python 端循环计算。
- 对零值做保护，避免除零异常。

10.3 CSV 导出

源码出处：`analytics/views.py`

```
response = HttpResponse(content_type="text/csv")
response["Content-Disposition"] = f'attachment; filename="{filename}"'
writer = csv.writer(response)
```

解析：

- 不依赖第三方导出库，原生实现轻量可靠。
- 便于答辩阶段展示“数据可导出、可复核”。

---

11. 演示数据工程（答辩友好）

11.1 一键造数命令

源码出处：`core/management/commands/seed_demo_data.py`

```
class Command(BaseCommand):
    help = "Seed demo users and business data for project demonstration."
```

解析：

- 将答辩演示前准备工作标准化为命令。
- 降低“每次演示环境不一致”的风险。

11.2 幂等设计

源码出处：`core/management/commands/seed_demo_data.py`

```
course, created = self._ensure_course(...)
chapter, _ = Chapter.objects.update_or_create(...)
lesson, _ = Lesson.objects.update_or_create(...)
```

解析：

- 反复执行不会无限新增重复数据。
- 非常适合答辩前“快速重置环境”。

---

12. 前端呈现技术（Django 模板）

12.1 母版模板 + 统一视觉变量

源码出处：`templates/base.html`

```
:root {
    --bg: #f3f6fb;
    --card: #ffffff;
    --text: #1d2530;
    --brand: #0f4c81;
}
{% block content %}{% endblock %}
```

解析：

- 采用母版模板统一导航、消息提示、视觉规范。
- 各页面只需填充 `block content`，复用性强。

12.2 服务端渲染（SSR）

- 页面由视图直接 `render(request, template, context)` 输出。
- 优势：技术栈统一、开发成本低、适合毕设快速迭代。

---

13. 测试与质量保证

13.1 测试覆盖关键业务链路

源码出处：`analytics/tests.py`、`core/tests.py` 等

```
response = self.client.get(reverse("analytics:index"))
self.assertEqual(response.status_code, 200)
self.assertContains(response, "个人概览")
```

```
call_command("seed_demo_data", skip_search_index=True)
self.assertTrue(Course.objects.filter(title="Python Full Stack 101").exists())
```

解析：

- 采用“模型测试 + 视图测试 + 命令测试”多层验证。
- 对答辩演示命令同样纳入测试，保障可复现性。

13.2 回归结果

在当前项目中已执行：

- `python manage.py check`
- `python manage.py test`

结果：84 个测试通过。

---

14. 关键设计取舍（可写入论文“方案论证”）

14.1 为什么采用 Django 全栈

- 单语言（Python）覆盖后端、模板、管理后台、脚本任务。
- 对毕设周期友好，开发效率高。

14.2 为什么搜索不直接多表 `icontains`

- 多表直接检索在数据增大后成本高。
- 采用 `SearchDocument` 去范式索引可提升检索可控性和扩展性。

14.3 为什么引入审计日志

- 内容系统涉及“创建、修改、上下线、归档”等关键动作。
- 审计日志为教学管理和问题追踪提供证据链。

14.4 为什么要有 seed 命令

- 毕设答辩中最怕“临场无数据/数据脏”。
- 命令化造数能显著提升演示稳定性与可重复性。

---

15. 论文可直接引用的“技术关键词”

- Django 单体模块化架构（Monolith + App 分层）
- Django ORM 领域建模与关系约束
- 基于角色的访问控制（RBAC 简化实现）
- 事务一致性（`transaction.atomic`）
- 去范式搜索索引（Denormalized Search Index）
- 信号驱动的索引同步（Event-driven by Django Signals）
- 幂等数据初始化命令（Idempotent Seeding）
- 指标聚合与数据导出（Analytics + CSV）
- 自动化测试驱动的质量保障（Test-backed Development）

---

16. 源码出处索引（便于论文引用）

- 项目配置：`config/settings.py`、`config/urls.py`
- 用户系统：`accounts/models.py`、`accounts/forms.py`、`accounts/views.py`
- 课程系统：`courses/models.py`、`courses/forms.py`、`courses/views.py`、`courses/audit.py`
- 测验系统：`quiz/models.py`、`quiz/forms.py`、`quiz/views.py`
- 论坛系统：`forum/models.py`、`forum/forms.py`、`forum/views.py`
- 搜索系统：`search/models.py`、`search/views.py`、`search/indexing.py`、`search/signals.py`、`search/management/commands/rebuild_search_index.py`
- 数据分析：`analytics/views.py`
- 演示命令：`core/management/commands/seed_demo_data.py`
- 测试样例：`analytics/tests.py`、`core/tests.py` 等
- 依赖清单：`requirements.txt`
- 模板基座：`templates/base.html`

---

17. 复现建议（论文附录可用）

1. `conda create -n learningwebsite python=3.11 -y`
2. `conda activate learningwebsite`
3. `pip install -r requirements.txt`
4. `python manage.py migrate`
5. `python manage.py seed_demo_data`
6. `python manage.py runserver`

访问：`http://127.0.0.1:8000/`